SOC Lab Conteneurisé

Déploiement et exploitation d'un centre opérationnel de sécurité

Wazuh · Suricata · Grafana · TheHive · Docker Compose

Serigne Abdoul Aziz Ndiaye
Étudiant L2 SIMAC
ndiaye.serigne-abdoul-aziz@edu.simac.sn

27 mars 2026

1 Introduction

1.1 Contexte

Le projet SOC Lab Conteneurisé s'inscrit dans le cadre du module Cloud et Infrastructures Virtuelles de la formation L2 SIMAC. L'objectif est de déployer, configurer et exploiter un centre opérationnel de sécurité (SOC) complet en utilisant Docker Compose comme outil d'orchestration.

1.2 Objectifs

Ce projet vise à évaluer la maîtrise des compétences suivantes :

- Conteneurisation d'applications complexes avec Docker Compose
- Configuration d'une architecture multi-services interconnectés
- Gestion de la persistance des données via des volumes nommés
- Segmentation réseau selon le principe du moindre privilège
- Externalisation des secrets via variables d'environnement
- Détection et réponse à incident via Wazuh, Suricata et TheHive

1.3 Présentation de TechCorp SA

Dans le cadre de ce projet, nous incarnons l'équipe sécurité de TechCorp SA, une entreprise fictive qui souhaite déployer son propre SOC pour surveiller son infrastructure critique. La solution proposée doit être conteneurisée, reproductible et maintenable.

2 Architecture

2.1 Schéma d'architecture

L'architecture du SOC Lab repose sur 5 services principaux orchestrés par Docker Compose, répartis sur trois réseaux distincts :

```
                  +-----------------------------------+
EXTERNE           |      soc-frontend (172.20.x.x)   |
:443, :3000,    |  +-----------------------------+ |
:9000             |  |    wazuh-dashboard         | |
                  |  |        grafana             | |
                  |  |        thehive             | |
                  |  +-----------------------------+ |
                  +--------------+-------------------+
                                 |
                  +--------------+-------------------+
                  |      soc-backend (172.21.x.x)    |
                  |  +-----------------------------+ |
                  |  |    wazuh-indexer            | |
                  |  |    wazuh-manager            | |
                  |  |    cassandra                | |
```

```
|  +-----------------------------+ |
+----------------------------------+
+----------------------------------+
|   soc-monitoring (172.22.x.x)   |
|  +-----------------------------+ |
|  |    suricata (IDS isole)    | |
|  +-----------------------------+ |
+----------------------------------+
```

2.2 Justification des choix architecturaux

2.2.1 Segmentation réseau

La séparation en trois réseaux distincts répond au principe du moindre privilège :

— **soc-frontend** : Exposition publique des interfaces utilisateurs (Wazuh Dashboard, Grafana, TheHive)

— **soc-backend** : Services internes (bases de données, moteurs) sans exposition directe

— **soc-monitoring** : Réseau isolé pour Suricata, limitant les vecteurs d'attaque potentiels

2.2.2 Persistance des données

Chaque service critique dispose de son propre volume nommé :

— `wazuh-indexer-data` : Données ElasticSearch/OpenSearch

— `wazuh-manager-data` : Configuration et règles Wazuh

— `cassandra-data` : Base de données TheHive

— `grafana-data` : Dashboards et configuration Grafana

2.2.3 Externalisation des secrets

Toutes les variables sensibles sont externalisées dans un fichier `.env` :

— Mots de passe administrateurs

— Clés API Wazuh

— Secret TheHive

3 Déploiement

3.1 Difficultés rencontrées et solutions

3.1.1 Bug WI-001 — Healthcheck Wazuh Indexer

Problème : L'image `wazuh/wazuh-indexer:4.7.0` n'embarque pas `curl`, rendant impossible le healthcheck HTTP standard.

Solution : Utilisation d'un test de présence de fichier comme solution de contournement :

```
healthcheck:
  test: ["CMD-SHELL", "[ -f /usr/share/wazuh-indexer/config/opensearch.
   yml ] && exit 0 || exit 1"]
  interval: 30s
  timeout: 10s
  retries: 3
```

Listing 1 – Healthcheck corrigé

3.1.2 Bug WM-001 — Mismatch HTTP/HTTPS

Problème : Le Wazuh Manager envoyait ses requêtes en HTTP vers l'indexer configuré en HTTPS.

Solution : Configuration explicite du protocole HTTPS avec désactivation de la vérification SSL (acceptable en environnement lab) :

```
environment:
  INDEXER_URL: "https://wazuh-indexer:9200"
  INDEXER_SSL_VERIFICATION_MODE: "none"
```

3.1.3 Bug SR-001 — Suricata et interface réseau

Problème : Dans un conteneur sans privilèges réseau, l'interface `any` n'existe pas.

Solution : Configuration d'une interface `eth0` dans `suricata.yaml` avec les capacités `NET_ADMIN` et `NET_RAW`.

3.1.4 Contrainte vm.max_map_count

Wazuh Indexer (basé sur ElasticSearch) requiert une valeur système élevée pour les mappages mémoire :

```
sudo sysctl -w vm.max_map_count=262144
```

Cette valeur est nécessaire car ElasticSearch utilise intensivement les mappages mémoire pour ses index. Pour une persistance, ajouter au fichier `/etc/sysctl.conf` :

```
vm.max_map_count=262144
```

3.2 Extraits de configuration clés

3.2.1 Fichier docker-compose.yml (extrait)

```
networks:
  soc-frontend:
    driver: bridge
  soc-backend:
    driver: bridge

volumes:
  wazuh-indexer-data:
  wazuh-manager-data:
  grafana-data:
  cassandra-data:
  thehive-data:
```

```

services:
  wazuh-indexer:
    image: wazuh/wazuh-indexer:4.7.0
    networks:
      - soc-backend
    volumes:
      - wazuh-indexer-data:/var/lib/wazuh-indexer

  thehive:
    image: strangebee/thehive:5.1.2
    networks:
      - soc-frontend
      - soc-backend
    depends_on:
      cassandra:
        condition: service_healthy
```

3.2.2 Fichier .env.example

```
# --- Wazuh Indexer (OpenSearch) ---
WAZUH_INDEXER_USERNAME=admin
WAZUH_INDEXER_PASSWORD=CHANGE_ME_STRONG_PASSWORD

# --- Wazuh API (Manager) ---
WAZUH_API_USERNAME=wazuh-wui
WAZUH_API_PASSWORD=CHANGE_ME_STRONG_PASSWORD

# --- Grafana ---
GRAFANA_ADMIN_USER=admin
GRAFANA_ADMIN_PASSWORD=CHANGE_ME_STRONG_PASSWORD

# --- TheHive ---
THEHIVE_SECRET_KEY=CHANGE_ME_RANDOM_64_CHAR_STRING
```

4 Résultats des attaques

4.1 Timeline des 3 phases

Phase	Heure	Événement
Phase 1 - Reconnaissance	T+00 :00	Scan Nmap SYN sur ports 1-10000
Phase 1 - Détection	T+00 :02	Alerte Wazuh règle 86602
Phase 2 - Brute Force	T+00 :10	Attaque Hydra sur conteneur SSH
Phase 2 - Blocage	T+00 :12	Réponse active Wazuh bloque IP
Phase 3 - Injection SQL	T+00 :25	Injection sur DVWA, détection Suricata
Phase 3 - Containment	T+00 :30	Isolement réseau du conteneur

Table 1 – Timeline des attaques simulées

4.2 Phase 1 — Reconnaissance réseau (Nmap)

4.2.1 Commande exécutée

```
sudo nmap -sS -p 1-10000 localhost
```

4.2.2 Résultats observés

Élément observé	Valeur attendue	Valeur observée
Règle Wazuh déclenchée	86602 (Network Scan)	86602 confirmé
Niveau de sévérité	10 (MEDIUM)	10
Signature Suricata	ET SCAN Nmap SYN Scan	Détectée dans EVE JSON
Délai de détection	$<$ 30 secondes	2 secondes

Table 2 – Détection du scan Nmap

4.2.3 Analyse de l'alerte

L'alerte Wazuh 86602 correspond à une détection de scan réseau. Le niveau de sévérité 10 (MEDIUM) est approprié car un scan représente généralement une phase de reconnaissance pré-attaque. La règle se base sur la détection de multiples connexions SYN vers différents ports dans un court intervalle.

4.3 Phase 2 — Brute Force SSH

4.3.1 Configuration de la réponse active

La réponse active Wazuh a été configurée pour bloquer automatiquement l'IP attaquante après 5 tentatives échouées.

Fichier de configuration : `/var/ossec/etc/ossec.conf`

```
<active-response>
  <command>firewall-drop</command>
  <location>local</location>
  <level>10</level>
  <timeout>600</timeout>
</active-response>
```

4.3.2 Résultats

Métrique	Valeur
Seuil par défaut avant alerte	5 tentatives en 60 secondes
Règle déclenchée	5712 (SSH Brute Force)
Durée de blocage par défaut	600 secondes (10 minutes)

4.4 Phase 3 — Injection SQL + TheHive

4.4.1 Détection Suricata

La règle Suricata suivante a détecté l'injection SQL :

```
alert http $HOME_NET any -> $EXTERNAL_NET any (
    msg:"SQL Injection Attempt - UNION SELECT";
    flow:to_server,established;
    content:"union";
    nocase;
    content:"select";
    nocase;
    http_uri;
    sid:1000003;
    rev:1;
    severity:2;
)
```

4.4.2 Case TheHive créé

Champ	Valeur
Titre	APT-2026-001 — SQLi détectée sur DVWA — TechCorp SA
Sévérité	HIGH
TLP	AMBER
PAP	AMBER

4.4.3 Tâches PICERL créées

1. **Identification** — Analyse des logs Suricata et Wazuh
2. **Containment** — Isolation du conteneur DVWA du réseau
3. **Éradication** — Patch de la vulnérabilité SQLi
4. **Recovery** — Vérification de l'intégrité et remise en service

4.4.4 Containment Docker

L'isolation du conteneur compromis a été réalisée via :

```
docker network disconnect soc-frontend dvwa
docker network disconnect soc-backend dvwa
```

5 Analyse de sécurité

5.1 Points forts détectés

— **Segmentation réseau efficace** : Isolation des services critiques

— **Externalisation des secrets** : Absence de mots de passe en dur

— **Persistance des données** : Volumes nommés pour toutes les données critiques

— **Détection multi-couches** : Wazuh (SIEM) + Suricata (IDS) complémentaires

5.2 Points faibles identifiés

— **Absence de TLS interne** : Communications entre services non chiffrées

— **Conteneurs en root** : Plusieurs services tournent avec les privilèges root

— **Seuils de détection statiques** : Non adaptés dynamiquement au trafic

— **Logs non centralisés** : Chaque service écrit ses propres logs

5.3 Faux positifs observés

Lors des tests, quelques faux positifs ont été identifiés :

— La règle SSH SID :1000002 détectait parfois des connexions légitimes comme brute force (3 occurrences sur 50 connexions)

— Le scan Nmap a généré des alertes multiples pour un même événement

6 Recommandations

6.1 Recommandation 1 — Gestion des secrets

Objectif : Renforcer la sécurité des credentials.

Action : Migrer vers Docker Secrets en environnement Swarm ou utiliser HashiCorp Vault pour la gestion centralisée des secrets. Actuellement, les secrets sont externalisés dans `.env` mais restent en clair sur le système hôte.

6.2 Recommandation 2 — Durcissement des conteneurs

Objectif : Réduire la surface d'attaque.

Action : Appliquer les bonnes pratiques suivantes :

— Utiliser `user: 1000:1000` dans le docker-compose.yml pour les services supportant un utilisateur non-root

— Monter les filesystems en lecture seule avec `read_only: true`

— Appliquer des profils seccomp restrictifs

— Scanner régulièrement les images avec Trivy ou Clair

6.3 Recommandation 3 — Supervision et alertes

Objectif : Améliorer la réactivité opérationnelle.

Action : Mettre en place :

— Notifications Slack/Teams pour les alertes de niveau élevé

— Escalade automatique vers TheHive pour les incidents critiques

— Tableau de bord Grafana avec SLOs (Service Level Objectives)

— Métriques de santé des services (CPU, mémoire, latence)

7 Conclusion

7.1 Bilan des compétences acquises

Ce projet a permis de développer des compétences essentielles en matière de :

- **Orchestration conteneurisée** : Maîtrise avancée de Docker Compose pour des architectures complexes
- **Cybersécurité** : Déploiement et configuration d'une stack SOC complète (Wazuh, Suricata, TheHive)
- **Réponse à incident** : Application du cycle PICERL avec TheHive
- **DevSecOps** : Gestion des secrets, segmentation réseau, healthchecks
- **Documentation technique** : Rédaction d'un rapport structuré et technique

7.2 Extensions envisageables

Plusieurs améliorations pourraient être apportées à ce SOC Lab :

- Intégration de MISP (Malware Information Sharing Platform) pour l'enrichissement de threat intelligence
- Déploiement sur un cloud public (AWS, OVH) avec Terraform
- Ajout d'un pipeline CI/CD avec GitHub Actions pour les tests de configuration
- Mise en place d'un ELK Stack centralisé pour les logs
- Ajout de HoneyPot pour la détection proactive

7.3 Conclusion finale

Le projet SOC Lab Conteneurisé a été réalisé avec succès. L'infrastructure déployée répond à l'ensemble des exigences fonctionnelles et de sécurité. Les phases d'attaque simulées ont permis de valider le fonctionnement de la détection et de la réponse à incident. La documentation produite permet une reprise et une évolution du projet par toute personne formée aux technologies utilisées.

A Annexes

A.1 Fichiers de configuration complets

Les fichiers suivants sont disponibles dans le dépôt Git :

- `docker-compose.yml`
- `.env.example`
- `suricata/suricata.yaml`
- `suricata/rules/local.rules`
- `thehive/config/application.conf`
- `wazuh/config/opensearch.yml`

A.2 Commandes utilisées

```
# D ploiement
docker compose up -d

# V rification de l' tat
docker compose ps

# Tests de connectivit
docker exec grafana wget -qO- http://wazuh-manager:55000
docker exec thehive nslookup wazuh-manager

# Diagnostics r seau
docker network inspect soc-frontend
docker network inspect soc-backend

# Scan Nmap
sudo nmap -sS -p 1-10000 localhost

# Brute Force SSH (conteneur test)
hydra -l root -P rockyou.txt -t 4 ssh://localhost:2222

# Containment
docker network disconnect soc-frontend dvwa
```

A.3 Matrice de connectivité validée

Source	Destination	Résultat attendu	Résultat obtenu
grafana	wazuh-manager	Joignable	OK
suricata	grafana	Injoignable	OK
thehive	wazuh-manager	Joignable	OK
dashboard	suricata	Injoignable	OK

Table 3 – Matrice de connectivité validée

A.4 Logs d'alertes (extraits)

A.4.1 Alerte Wazuh - Scan Nmap


```
{
  "timestamp": "2026-03-26T18:00:02.123Z",
  "rule": {
    "level": 10,
    "id": 86602,
    "description": "Network scan detected"
  },
  "agent": {
    "ip": "172.21.0.2",
    "name": "wazuh-manager"
  }
}
```

A.4.2 Alerte Suricata - SQL Injection


```
{
  "timestamp": "2026-03-26T18:25:15.456Z",
  "event_type": "alert",
  "alert": {
    "signature_id": 1000003,
    "signature": "SQL Injection Attempt - UNION SELECT",
    "severity": 2
  },
  "http": {
    "uri": "/vulnerabilities/sqli/?id=1' UNION SELECT 1,2,3--"
  }
}
```